Understand Anything: codebase graphs, onboarding and incremental verification

Researched 2026-10-20.

What it offers

Understand Anything turns repository structure into a navigable graph with explanations, tours and questions. The current project README (https://github.com/Egonex-AI/Understand-Anything) describes deterministic parsing plus model-generated semantic material. Treat graph structure and generated explanations as different evidence: a plausible narrative can still misstate behavior.

Current install and first run

The documented Claude Code route is:

```
/plugin marketplace add Egonex-AI/Understand-Anything
/plugin install understand-anything
/understand src
/understand-dashboard
/understand-chat How does the payment flow work?
/understand-diff
```

Choose a small source directory for the first scan. Inspect the plugin before allowing repository analysis, and understand where the selected model sends source content. No installation or scan was executed for this guide.

The project documents a .ua knowledge graph, with a legacy data-directory fallback. The first scan may use substantial tokens; incremental runs reduce the amount reanalyzed, but the magnitude must be measured on the chosen repository. Installation, analysis and sharing (https://github.com/Egonex-AI/Understand-Anything#quick-start).

Share a generated graph

The documented read-only viewer requires Node.js 18+ and an already analyzed project:

```
npx https://github.com/Egonex-AI/Understand-Anything/releases/latest/download/understand-anything-viewer.tgz /path/to/analyzed/project
```

Replace the example path. This downloads and runs the viewer; it is a command for later use, not one executed here. Pin and inspect a release if reproducibility matters. A local viewer does not imply that the earlier semantic generation was local. Check generated graph content before sharing it, because summaries and code excerpts can disclose private implementation details.

Evaluate usefulness, not node count

Pick five onboarding questions whose answers you can establish from code, such as how a request reaches storage, which code validates input and where a callback is registered. Record answer correctness with file references and time-to-answer. Then rename one file, change an import and remove a temporary symbol; rerun the analysis and inspect whether the graph reflects each change.

Review a random sample of edges against actual imports and call sites. Mark an inference as an inference; a graph should not turn every association into a confirmed dependency. Save scan duration, files analyzed, model usage and wrong explanations. Compare with a plain repository search baseline.

Reusable prompt

```
Evaluate Understand Anything on a small approved source directory.
Before scanning, report model destination, estimated scope and generated files.
Use five answerable onboarding questions with independently verified file paths.
Compare first and incremental runs after a rename, import change and deletion.
Report wrong edges and unsupported summaries, not just graph size.
Do not publish source or generated graphs.
```

Project idea: onboarding evidence board

Build a small companion board with question, cited path, answer, reviewer verdict and elapsed time. Link each card to a graph node when available. First ship static JSON plus a read-only viewer, then add import of actual scan reports. Acceptance: another reader can follow each accepted explanation back to the code.

Relationship to existing Kiln topics

Code storylines and semantic search are useful neighbors, but this is a specific graph tool with its own install and artifacts. Keep it independently discoverable rather than merge it merely because both improve code comprehension. Star counts and supported-platform totals change and are not an adoption test.

Original sources

- https://www.instagram.com/reel/Deiu2taCLDN/